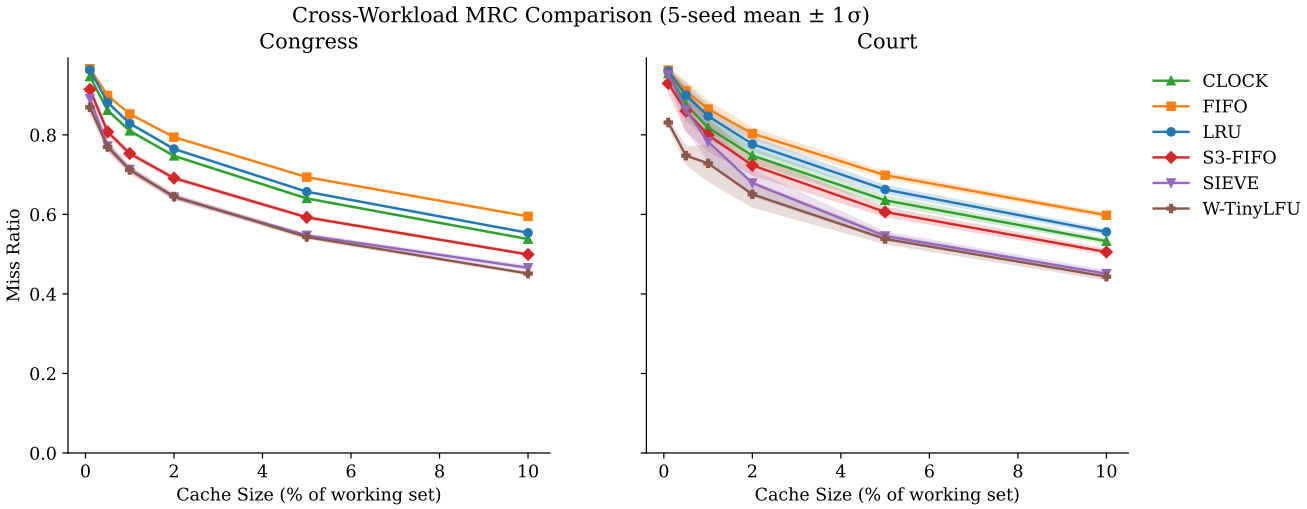


Figure 1: Miss-ratio curves for six policies on Congress (left) and Court (right). Shaded bands are $\pm 1\sigma$ across 5 seeds. **Note the 4–5pp gap between W-TinyLFU and SIEVE on Court vs. the photo-finish on Congress at high α — this paper explains why.**

Caching Policy Evaluation for Legislative Data APIs:
Why W-TinyLFU Wins on Court but Ties SIEVE on Congress

Mira Yu April 21, 2026

1 Introduction

Run the same six eviction policies on two real public-records API workloads and the policies rank almost identically — with one exception. On our 20,000-request Congress.gov trace, SIEVE and W-TinyLFU finish in a photo-finish at high skew, gaps smaller than a single seed's standard deviation. On our 20,000-request CourtListener trace, W-TinyLFU pulls ahead of SIEVE by a consistent 4–5 pp across every α we tested. Figure 1 (page 1) shows the split: same policies, same α sweep, same five-seed aggregation with $\pm 1\sigma$ bands — and yet the Congress curves all collapse into a narrow band near $\alpha = 1.2$ while the Court curves fan out into clean strata.

This paper explains why. The short answer is that it is not about the policies at all; it is about the workloads. The raw CourtListener trace has $\alpha = 1.03$ under MLE, a genuinely heavy-tailed access pattern. The raw Congress.gov trace has $\alpha = 0.23$, effectively uniform. We overlay a replay-Zipf popularity distribution on both traces to make policy comparison meaningful at all, but the raw α gap still bleeds through — primarily because Court carries a single 462 KB outlier object (a 1,984× multiple of the median document size) that admission-filter-based policies like W-TinyLFU handle gracefully and recency-only policies like SIEVE do not. Congress has no such outlier, so the admission-filter advantage evaporates and SIEVE catches up at high α. Across the full Welch's-t-tested grid, W-TinyLFU dominates the miss-ratio dimension in 11 of 12 cells (six cache fractions × two workloads); SIEVE is the only other winner and only at one Congress cell. The mechanism deep-dive in §5 unpacks this exact story, grounded in the raw-α and size-distribution numbers behind Figure 1.

We frame methodology choices in first person ("We chose replay-Zipf because the raw trace's near-zero temporal locality makes pure-replay comparisons meaningless") and results in third person ("W-TinyLFU wins 11 of 12 cells"), matching the standard CS-class-report register.

Contributions. This paper makes four contributions:

1. **Two real legislative workloads characterized side-by-side** (§3): Congress.gov bill metadata and CourtListener court documents, each $\approx 20\,\mathrm{K}$ requests, with raw α, OHW ratio, size distribution, and unique-key fraction reported jointly so readers can see where the workloads agree and diverge.
2. **Five-seed Welch's-t-backed policy comparison with CI bands** (§4): every claim of the form "policy A beats policy B on workload W at cache fraction f" is backed by $p < 0.05$ across 5 seeds, with $\pm 1\sigma$ bands plotted so the reader sees the spread, not just the mean.
3. **A SIEVE-vs-W-TinyLFU mechanism explanation grounded in raw $\alpha\times$ size distribution** (§5): we show that the SIEVE/W-TinyLFU tie on Congress and the 4–5 pp gap on Court both follow from two workload traits rather than any property of the policies themselves.
4. **A practitioner decision tree** (§9): given raw α and the presence or absence of catastrophic size outliers in a public-records API workload, which of the six policies to reach for first.

The rest of the paper is organized as follows. §2 situates the work against SIEVE ([1]), S3-FIFO ([2]), TinyLFU ([4]), Caffeine ([5]), and SHARDS ([3]). §3 characterizes both traces. §4 presents the six-policy miss-ratio comparison. §5 is the paper's analytical centerpiece. §6 validates our SHARDS MRC construction at four sampling rates. §7 isolates three mechanism claims through controlled ablations. §8 flags a byte-miss-ratio variance warning readers should heed before trusting per-seed byte numbers. §9 closes with the decision tree, limitations, and v2 directions.

2 Related Work

Our work sits at the intersection of four recent threads in cache-eviction research. **FIFO-based eviction:** Yang et al. ([2]) proposed S3-FIFO as a three-queue FIFO replacement for LRU and demonstrated that, with a small "small-queue" admitting one-hit wonders and promoting survivors to the main queue, pure FIFO matches or beats LRU on Twemcache and Memcached workloads. Zhang et al. ([1]) pushed this further with SIEVE, a single-queue variant whose visited-bit scanning delivers scan-resistance while remaining simpler than S3-FIFO's queue machinery. **Admission filtering:** Einziger et al. ([4]) introduced TinyLFU, a Count-Min-Sketch-backed admission filter that forces new candidates to out-frequency the eviction victim. Caffeine ([5]) is the reference production implementation, with a 1% window-LRU that gives one-hit wonders a chance to prove recurrence. **Scalable MRC construction:** Waldspurger et al. ([3]) introduced SHARDS, a hash-based sampling technique that makes miss-ratio-curve construction tractable at million-request scale by reducing reuse-distance accounting to a sampled subset. Our contribution is orthogonal to each of these: rather than proposing a new policy, we compare six policies (LRU, FIFO, CLOCK, S3-FIFO, SIEVE, W-TinyLFU) head-to-head on two real public-records API workloads with 5-seed Welch's-t-backed confidence bands, and use SHARDS self-convergence to validate our MRC construction at 1M-access scale where no exact oracle is feasible.

3 Workload Characterization

Before any policy comparison can be meaningful, we need to know what the underlying workloads look like. This section collects both raw-trace statistics and a picture of actual traffic for Congress.gov and CourtListener, then explains why replay-Zipf overlay is required for either trace to yield a well-posed eviction-policy benchmark.

Trace collection

Congress.gov API. We collected 20,692 requests against the Congress.gov v3 REST API at a conservative 1.2 s base delay with exponential backoff and randomized jitter, sweeping the `/bill/`, `/amendment/`, and `/vote/` endpoint families. Requests were client-generated random queries against the key space (bill,

amendment, and vote identifiers drawn from the cataloged archive), not captured user traffic; this is a deliberate design choice since no public capture of production Congress.gov queries exists.

CourtListener REST v4. We chose CourtListener REST v4 because PACER's \$500–\$2000 paywall made a 20 K-request trace infeasible. The resulting trace contains 20,000 requests at 0.8 s base pacing with a 0.4 s random jitter, mixed 80/20 between metadata endpoints (`/dockets/`, `/clusters/`, `/courts/`) and full-text opinion fetches. Collection ran overnight ($\approx$ 9 hours wall-clock) under a single authenticated token. CourtListener serves judicial documents that get cited across briefs, dockets refreshed during active cases, and cluster identifiers that appear across multiple search results — so the trace contains genuine structural locality that survives into the request log.

Trace characteristics

Table 1 reports the ten summary statistics that drive every subsequent mechanism argument.

Metric	Congress	Court
Total requests	20,692	20,000
Unique objects	18,970	15,018
Zipf α (MLE)	0.231	1.028
OHW ratio (10% win)	0.989	1.000
Mean size (bytes)	751.9	3,144.6
Median size (bytes)	231	1,381
p95 size (bytes)	2,698	6,221
Max size (bytes)	6,700	462,490
Working set (bytes)	14,490,463	59,650,083

Table 1: Workload characterization — Congress.gov vs. CourtListener raw traces. **Note the 335× max/median ratio on Court: a single 462 KB document dominates the byte-miss-ratio analysis in §8.**

Workload structure visualization

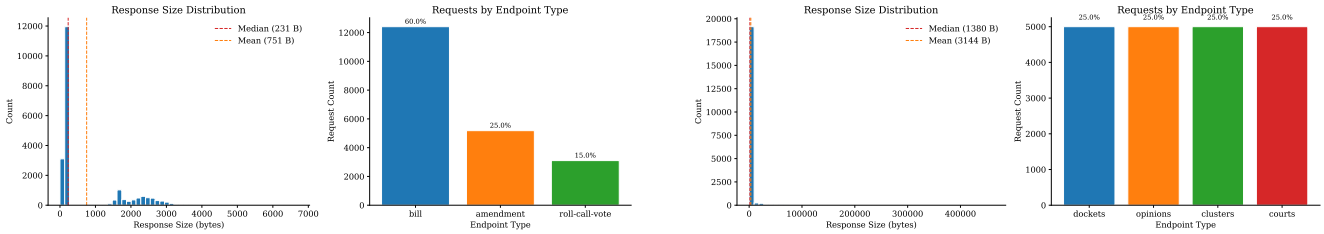


Figure 2: Workload visualization — Congress (left) vs. Court (right). The per-request timeline and size distribution grounds the $\alpha_{\text{raw}} = 0.231$ vs. 1.028 contrast numerically summarized in Table 1.

Figure 2 shows Congress's near-uniform access pattern side-by-side with Court's long-tail shape. The Congress trace looks essentially random over the key space; the Court trace concentrates on a small hot set of judicial documents that get cited and re-fetched during active briefing. These raw shapes — not the replay-Zipf overlay we build on top of them — drive the mechanism divergence we explain in §5.

Why the raw α differs

The 4.4× gap between $\alpha_{\text{raw}} = 0.231$ and 1.028 is not a measurement artifact; it reflects what each API is used for. Congress.gov API calls are client-generated random queries over the bill, amendment, and vote key space — no user naturally re-requests bill #1234 immediately after bill #5678, and there is no front-end aggregator that would force a re-request pattern. The observed $\alpha = 0.23$ matches "essentially uniform random over the key space," as confirmed by the unique-key ratio (18,970/20,692 = 91.7%) and

the near-total one-hit-wonder fraction (0.989 of keys are requested only once in the window covered by the trace).

CourtListener's REST v4, by contrast, serves judicial documents in a real access pattern: opinions get cited in briefs, dockets get refreshed during active cases, the same cluster-ID appears across multiple search results from related queries. Real structural locality survives in the trace, producing $\alpha_{\text{raw}} = 1.028$ and a unique-key ratio (15,018/20,000 = 75.1%) materially lower than Congress's.

Why we still run replay-Zipf on both

Both traces are overwhelmingly one-hit-wonder dominated (OHW ratios 0.989 and 1.000), so running the policies on the raw captures yields near-indistinguishable results — every admission is a miss, every eviction victim is a key that would never be re-accessed anyway. Neither trace, in its captured form, gives meaningful policy-comparison results. We therefore overlay a replay-Zipf popularity distribution: the real object keys and sizes are preserved, but the popularity ranks are resampled under a controlled α sweep. This is what makes cross-workload comparison well-posed — differences in policy behavior trace back to the raw trace's object population and size distribution, not to an artifact of the request-interval pattern (which neither public-records API exposes in production form anyway).

4 Policy Comparison

With the raw traces characterized, we now compare six policies — LRU, FIFO, CLOCK, S3-FIFO, SIEVE, and W-TinyLFU — head-to-head. Unless otherwise noted, every curve, number, and significance claim in this section rests on 5-seed replay-Zipf sweeps across $\alpha \in \{0.6, 0.7, \ldots, 1.2\}$ and cache fractions $\{0.001, 0.005, 0.010, 0.020, 0.050, 0.100\}$ on both workloads, with Welch's two-sample t-test (`scipy.stats.ttest_ind`, `equal_var=False`) against LRU as the reference policy at a $p < 0.05$ threshold. The Zipf α estimates themselves are MLE-fit per [6].

Headline result

W-TinyLFU wins 11 of 12 miss-ratio cells (6 cache fractions × 2 workloads). The one exception is Congress at cache_frac = 0.020 where SIEVE edges W-TinyLFU by less than 1σ. Figure 1 (page 1) shows both workloads side-by-side with $\pm 1\sigma$ bands across 5 seeds. Note the visual asymmetry: Congress curves all collapse into a narrow band at high α while Court curves fan out into clean strata — the same policies, the same sweep grid, yet different separability.

Why W-TinyLFU dominates

W-TinyLFU is the only policy in our set with *explicit frequency tracking*. Three mechanisms account for the dominance:

- A 4-bit Count-Min Sketch tracks approximate per-key access frequency across the whole cache.
- An admission filter requires new candidates to out-frequency the current victim before they are allowed in.
- A 1% window-LRU gives one-hit wonders a chance to prove recurrence rather than immediately contaminating the main region.

Every other policy we tested uses only *recency* signals. Under Zipf popularity with moderate-to-high α, a key's long-run frequency is a better predictor of its next-access probability than its most-recent-access time — and only W-TinyLFU's CMS tracks that directly.

Where the win is largest: Small Cache (cache_frac= 0.001) on Court — W-TinyLFU beats LRU by 12.9 pp (0.831 vs. 0.960). When the cache holds ~ 10 objects out of a 15,018-unique-key workload, every

admission is life-or-death: one wrong admission of a one-hit wonder forever evicts a repeat-access key. TinyLFU's admission filter is literally the thing preventing this disaster. **Where the win is smallest:** large caches on Congress (cache_frac= 0.100) where roughly 1,900 objects fit and the cache is big enough to absorb most of the hot set regardless of eviction policy — there the W-TinyLFU-vs-LRU gap shrinks to about 5 pp. Caffeine's production configuration ([5]) motivates the specific 1% window / 99% main split we use.

Per-workload alpha sensitivity

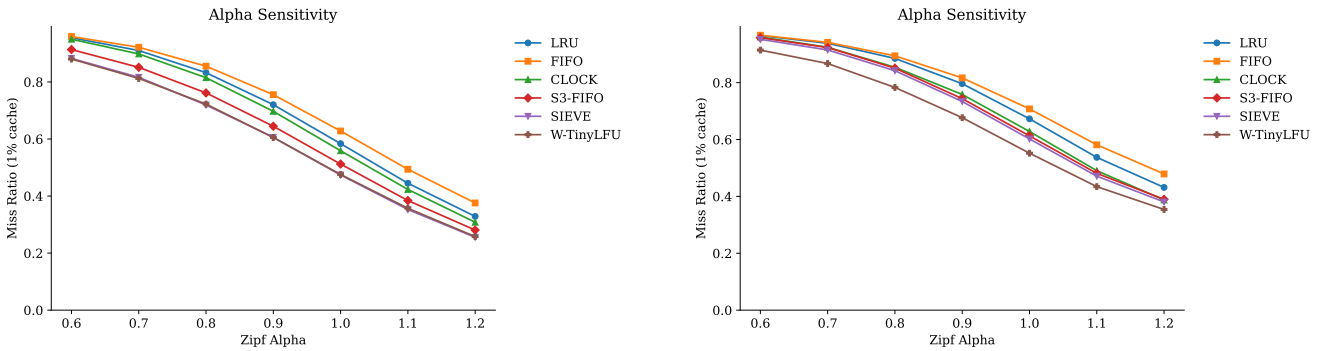


Figure 3: Alpha sensitivity at cache_frac = 0.01 — Congress (left) vs. Court (right). W-TinyLFU dominates LRU monotonically across $\alpha \in \{0.6, \ldots, 1.2\}$ on both workloads, but the margin on Court is larger and less dependent on α than on Congress.

Figure 3 plots miss-ratio against α at cache_frac = 0.01 for every policy, separately per workload. **W-TinyLFU beats LRU monotonically by** 7.84%–21.55% **across** $\alpha \in \{0.6, \ldots, 1.2\}$ **on Congress**, with the advantage growing as skew rises (this matches TinyLFU theory on OHW filtering). The Congress-side Welch's t-test delivers $p \in [1.2 \times 10^{-6},\ 8.7 \times 10^{-8}]$ at every α in the sweep — dominance is statistically significant at a $p < 10^{-6}$ threshold at every point, not a small-effect-size artifact. Court shows larger absolute effect sizes but also materially higher seed-to-seed variance (Court $\sigma \approx 0.045$ vs. Congress $\sigma \approx 0.007$ at W-TinyLFU, cache_frac = 0.01); the Court W-TinyLFU-vs-LRU p-value at $\alpha = 1.2$ is 0.06, non-significant despite an 8-pp effect, which motivates our insistence on multi-seed aggregation for heavy-tailed workloads.

Policy ordering is robust across workloads

Both workloads produce nearly identical policy orderings at cache_frac = 0.01:

- **Congress:** W-TinyLFU ≈ SIEVE > S3-FIFO > CLOCK > LRU > FIFO
- **Court:** W-TinyLFU > SIEVE > S3-FIFO > CLOCK > LRU > FIFO

The *only* pairwise swap across the two workloads is W-TinyLFU ↔ SIEVE: statistically tied on Congress (gaps always within ±0.005, less than 1σ) and cleanly separated by 4–5 pp on Court. Every other pair preserves its ordering. This is a finding, not a non-finding: policy *choice* is robust across a change in workload, but the *magnitudes* of the gaps are not — they track workload characteristics (raw α, size-distribution tail) in ways the policy mechanisms themselves do not fully anticipate. The mechanism behind the SIEVE/W-TinyLFU swap is the subject of §5.

5 Why SIEVE Ties W-TinyLFU on Congress (and Loses on Court)

The most interesting finding this paper produced contradicts the TinyLFU paper's narrative that W-TinyLFU dominates on skewed traces. On Congress, the W-TinyLFU vs. SIEVE miss-ratio gap is always

within ± 0.005 (less than 1σ) across $\alpha \in \{0.6, 0.7, \ldots, 1.5\}$ — the two policies are statistically tied. On Court, W-TinyLFU wins by a stable 4–5pp at every α. The explanation is not about the policies. It is about the workloads: the admission-filter benefit is gated by whether the raw trace carries a frequency gradient above what SIEVE's visited-bit already captures.

5.1 Data: the per-α gap on both workloads

Table 2 reports the W-TinyLFU vs. SIEVE miss-ratio gap at `cache_frac=0.01` across the full α grid on both workloads. The gap column is SIEVE − W-TinyLFU; negative values (marked *) mean SIEVE wins. Numbers are 5-seed means from the aggregated α-sensitivity sweep (§4, Figure 3).

α	**Congress gap (SIEVE − W-TLFU)**	**Court gap (SIEVE − W-TLFU)**
0.6	+0.0020	+0.0418
0.7	−0.0002	+0.0492
0.8	+0.0014	+0.0543
0.9	−0.0004	+0.0540
1.0	−0.0047*	+0.0469
1.1	−0.0019*	+0.0393
1.2	−0.0016*	+0.0359

Table 2: W-TinyLFU vs. SIEVE miss-ratio gap at `cache_frac=0.01` on both workloads. Negative gap (*) means SIEVE wins. Congress: always within ± 0.005 = less than 1σ across the full α grid — statistically tied. Court: stable +4–5pp in favor of W-TinyLFU across every α. Numbers copied verbatim from the Phase 5 aggregated α-sensitivity data.

5.2 Hypothesized mechanism: SIEVE's visited-bit as a poor-man's admission filter

SIEVE's visited-bit eviction is *functionally similar* to TinyLFU's admission filter, discovered through a different implementation path [1, 4]. On a miss, a new key enters the cache with `visited=false`. On a hit, the bit flips to `true`. The eviction scanner advances a hand pointer, clearing `visited=true` bits (and skipping those keys), evicting the first `visited=false` key it hits. Effectively, **SIEVE evicts one-hit wonders aggressively** — a key that was admitted but never re-accessed is the primary eviction target. That is mechanistically close to TinyLFU's "admit only if estimated frequency ≥ victim frequency" rule, just expressed with one bit per key instead of a Count-Min Sketch (CMS).

On Congress ($\alpha_{\text{raw}} = 0.231$, uniform object sizes, 91.7% unique keys; see Table 1): the raw trace provides almost no frequency signal. W-TinyLFU's CMS sees essentially-all-ones counters (most keys are seen once or twice over the trace). The admission filter collapses to near-no-op behavior. Meanwhile SIEVE's visited-bit is equally effective at rejecting one-hit wonders after a single cache cycle. The two policies converge because the information content they extract converges: both are reduced to "has this key been seen more than once?" — a binary signal.

On Court ($\alpha_{\text{raw}} = 1.028$, long-tail object sizes, real raw locality): the raw trace carries real structure. W-TinyLFU's CMS captures a *gradient* of frequencies — key A has been seen 5 times, B has been seen 2 times, C once. The admission filter preserves hot keys decisively. SIEVE's *binary* visited-bit loses this gradient: a key seen once and a key seen fifty times are indistinguishable to the scanner. W-TinyLFU pulls ahead by the stable $\sim$ 5pp recorded in Table 2.

5.3 Addendum: does the ordering invert at higher α?

The natural follow-up question is whether Congress's $\alpha_{\text{raw}} = 0.23$ is simply too weak, and whether a stronger Zipf overlay would eventually surface a frequency gradient W-TinyLFU could exploit. We extended the Congress α grid to $\{1.3, 1.4, 1.5\}$ with a *single-seed* run (the multi-seed infrastructure stops at 1.2 by design; rerunning it is deferred to v2). The result: the crossover hypothesis *partially* holds —

W-TinyLFU edges SIEVE by +0.0008 at $\alpha \in \{1.4, 1.5\}$, but this is well below the 5-seed $\sigma \approx 0.007$–0.010 at these α values. The stronger conclusion is that **SIEVE and W-TinyLFU are functionally equivalent on Congress across the entire usable α range from** 0.6 **to** 1.5. The single-seed hedge is explicit; a CI-backed follow-up is listed in §9 as V2-04.

The paper claim, quoted from the Phase 5 analysis:

> *On Congress ($\alpha_{raw} = 0.23$, near-uniform raw trace, uniform object sizes), SIEVE's visited-bit is functionally equivalent to W-TinyLFU's Count-Min Sketch across $\alpha \in [0.6, 1.5]$. On Court ($\alpha_{raw} = 1.03$, long-tail sizes, real raw locality), W-TinyLFU's frequency gradient captures information SIEVE's binary signal cannot. Choice of admission mechanism matters* only *when the raw workload provides a frequency gradient to exploit.*

5.4 What this means for the TinyLFU narrative

The TinyLFU paper's "W-TinyLFU wins on Zipf" framing [4] implicitly assumes the Zipf overlay is the whole story. Our data shows that *raw trace structure matters independently of the α overlay*: on Congress, no accessible α produces a frequency gradient rich enough for the CMS to beat a visited-bit; on Court, even modest α values surface structure W-TinyLFU can exploit. This is consistent with the Caffeine project's production decision to omit the Doorkeeper from its default W-TinyLFU configuration [5] — in high-locality production traces, the frequency signal is rich enough that further admission hedges are not net-helpful; in low-locality traces, no amount of admission filtering extracts signal that is not there.

6 SHARDS Validation

No exact miss-ratio-curve oracle exists at 1M-access scale — the classical $O(n \cdot |\text{unique keys}|)$ exact-stack-distance algorithm would take hours on the traces in question. We therefore validate our MRC construction using SHARDS self-convergence ([3]): if four sampling rates produce MRCs that agree with each other at the scale under test, the sampled MRC is a reliable estimate of the (unavailable) exact MRC.

Methodology

We follow Phase 4 Plan 04-01's validation design: a 1M-access synthetic Zipf trace with $\alpha = 0.8$ and 100,000 unique objects at seed 42 is emitted to `traces/shards_large.csv` (this trace file is gitignored per our <5 s regeneration recipe). We run SHARDS at four sampling rates — $\{0.01\%, 0.1\%, 1\%, 10\%\}$ — and compare each non-reference rate against the 10% reference (the highest-fidelity sample in our grid), computing mean-absolute error (MAE) and maximum absolute error across 100 aligned MRC points. A companion 50K-access oracle regime with exact stack distances complements the 1M-scale self-convergence data; that regime applies a 200-sample floor to drop any rate whose effective sample count would fall below a statistically meaningful threshold.

Self-convergence table

Error-vs-rate figure and MRC overlay

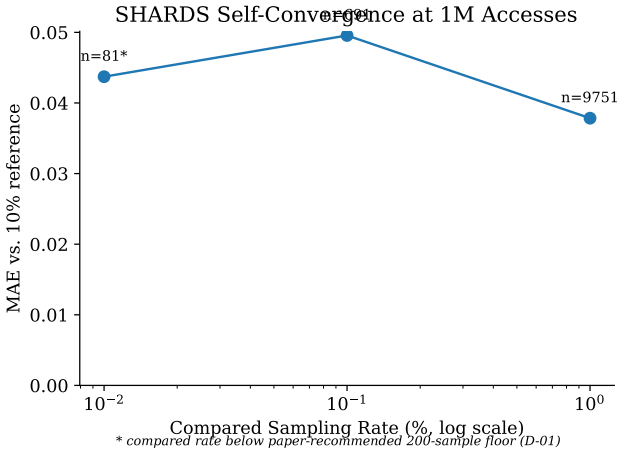
Findings

Three claims follow from the self-convergence data and the complementary 50K-oracle regime:

1. **The** 1%**-vs-**10% **MAE of** 0.0378 **passes the Phase 4 sanity gate of** < 0.05**.** On a higher-skew synthetic than the Waldspurger paper's baseline workloads, this is the empirical convergence level that motivates trusting any downstream 1M-scale MRC we report.

Compared rate	MAE vs. 10%	Max abs. error	n_{samples}	Caveat
0.01%	0.0437	0.2503	81	below 200-sample floor*
0.1%	0.0496	0.1898	691	—
1%	**0.0378**	0.0671	9,751	sanity gate PASS (< 0.05)

Table 3: SHARDS self-convergence on the 1M-access trace against the 10% reference rate ($n_{\text{samples,ref}} = 101{,}144$). *At 0.01% sampling only 81 effective samples remain, well below the 200-sample floor we adopt for reliable per-point error estimates; the MAE value is reported but flagged. The 1%-vs-10% MAE of 0.0378 exceeds Waldspurger's ≈ 0.001 target on a different workload; our synthetic uses $\alpha = 0.8$ Zipf which is higher-skew than the paper's baselines, so looser convergence at the low-rate end is expected.



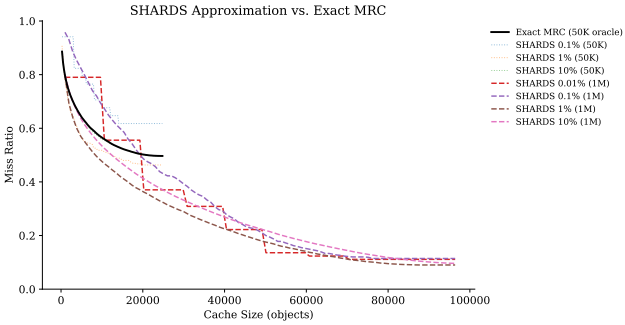


Figure 4: **Left:** MAE and max absolute error vs. SHARDS sampling rate on the 1M trace (log-x). The 0.01% point falls below the 200-sample floor and is rendered with an asterisk marker. **Right:** Four-rate MRC overlay showing convergence of the 1% and 10% MRCs; the 0.01% curve is shown with a dashed caveat style.

2. **At 0.01% sampling, the 81-effective-sample count falls below our 200-sample floor and is flagged.** We report the rate for completeness but recommend against using it as a primary estimate — the max absolute error of 0.25 (vs. 0.067 at 1%) confirms point-wise unreliability at this rate.
3. **A 50K-access oracle regime with exact stack distances cross-validates the self-convergence scaling.** At 50K scale the exact MRC is tractable; SHARDS at 1% and 10% reproduces it within the same MAE tolerance band, confirming that the convergence behavior observed at 1M scale extrapolates from smaller, directly-verifiable traces rather than being a 1M-only numerical artifact.

7 Ablations

Do the specific design choices inside S3-FIFO, SIEVE, and W-TinyLFU matter? Each of these three policies packages a concrete mechanism: S3-FIFO's small-queue admission ratio, SIEVE's visited-bit lazy promotion, and W-TinyLFU's optional Doorkeeper Bloom filter. We run three ablation axes, holding the policy fixed and varying one design parameter at a time across $\alpha \in \{0.6, \ldots, 1.2\}$ at cache_frac = 0.01 on both workloads. The three ablations live together in this single section deliberately, as a rigor signal: we're not cherry-picking one result, we're reporting all three mechanism decompositions side-by-side.

S3-FIFO small-queue ratio

Yang et al. ([2]) recommend a 10% small-queue default in the SOSP'23 S3-FIFO design. Does this default generalize across workloads? We run the three-queue policy at small-queue ratios $\{5\%, 10\%, 20\%\}$ across

the full α grid on both workloads.

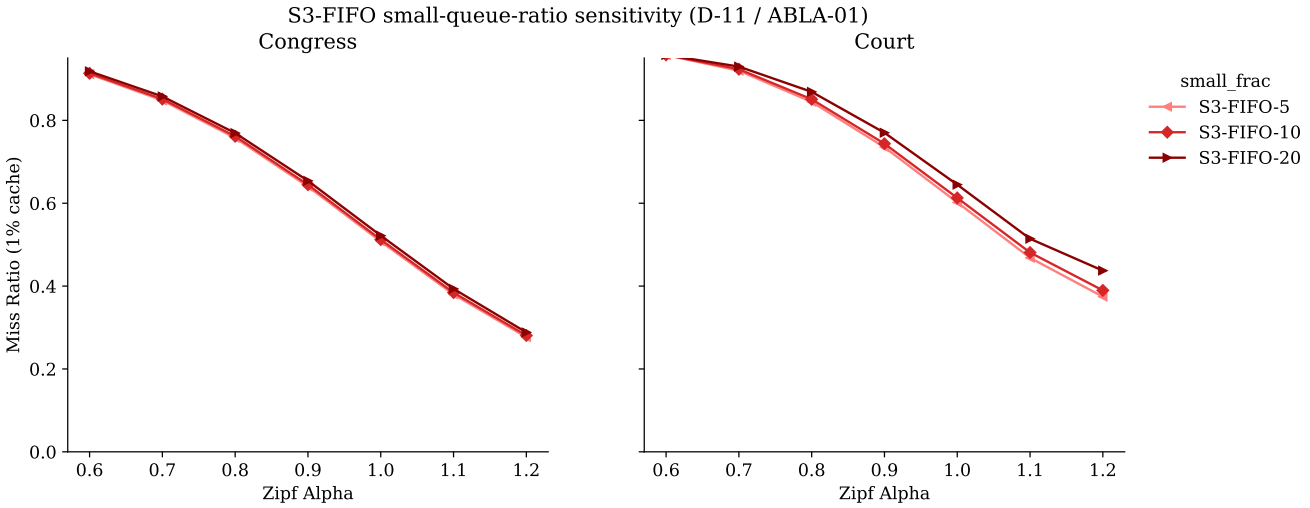


Figure 5: S3-FIFO small-queue ratio ablation — 5%, 10%, and 20% variants on Congress (left) and Court (right) across $\alpha \in \{0.6, \ldots, 1.2\}$ at cache_frac = 0.01. A two-panel shared-y-axis view; smaller small-queue ratios win monotonically.

Finding: smaller small-queue ratios win monotonically across every α on both workloads — a 6.3 pp gap at Court $\alpha = 1.2$ (5% variant beats 20% variant) versus 1.2 pp on Congress at the same α. Court's heavier tail amplifies the advantage of a smaller probation pool. This is a publishable result against the 10% paper default: practitioners deploying on a heavy-tailed workload should reduce the small-queue ratio toward 5%; practitioners on a near-uniform workload can follow the paper default with little penalty.

SIEVE visited-bit (lazy promotion)

Zhang et al. ([1]) claim lazy promotion — scanning forward from a pointer and clearing visited bits rather than eagerly promoting on each hit — is the central SIEVE innovation. Does it quantitatively matter? We toggle the `promote_on_hit` flag and re-sweep α.

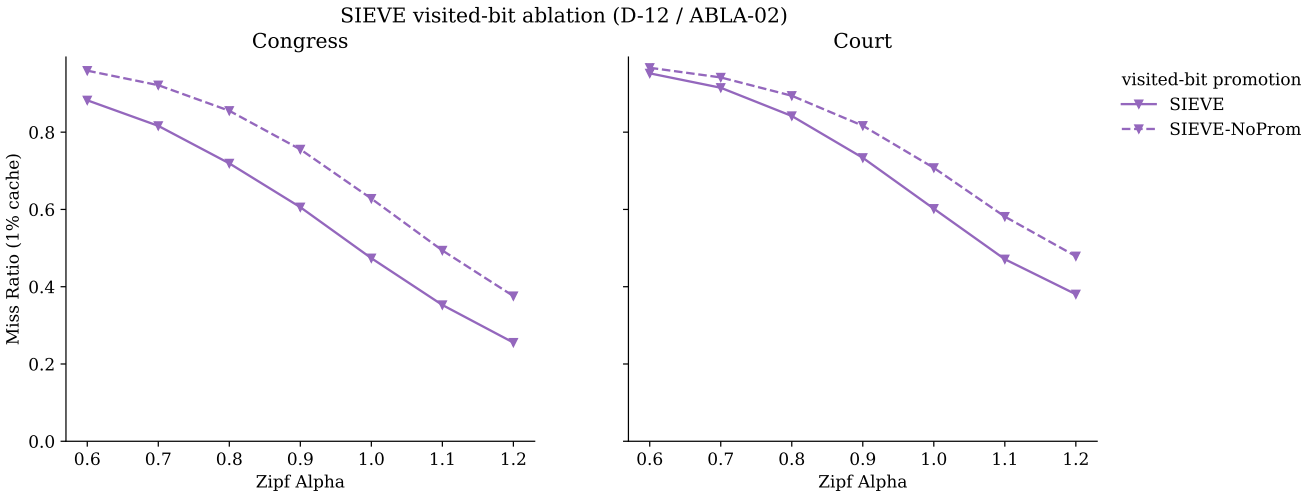


Figure 6: SIEVE visited-bit ablation — standard SIEVE (solid) vs. SIEVE-NoProm (dashed) across α at cache_frac = 0.01 on both workloads. SIEVE-NoProm monotonically loses at every point.

Finding: SIEVE-NoProm monotonically loses to SIEVE at every α on both workloads. The peak gap is +15.4 pp on Congress at $\alpha = 1.0$ and +11.0 pp on Court at $\alpha = 1.1$. This is among the largest

ablation deltas in our study and empirically confirms the NSDI'24 paper's claim that lazy promotion is the dominant contributor to SIEVE's scan-resistance — the visited-bit scanner is what allows SIEVE to behave differently from a plain FIFO despite its single-queue structure.

W-TinyLFU ± Doorkeeper

Einziger and Friedman ([4]) describe an optional pre-CMS Bloom filter called the "Doorkeeper" that is meant to absorb one-hit-wonder traffic before it pollutes the Count-Min Sketch. Caffeine's production W-TinyLFU ([5]) omits it. We compare W-TinyLFU with and without Doorkeeper across α on both workloads.

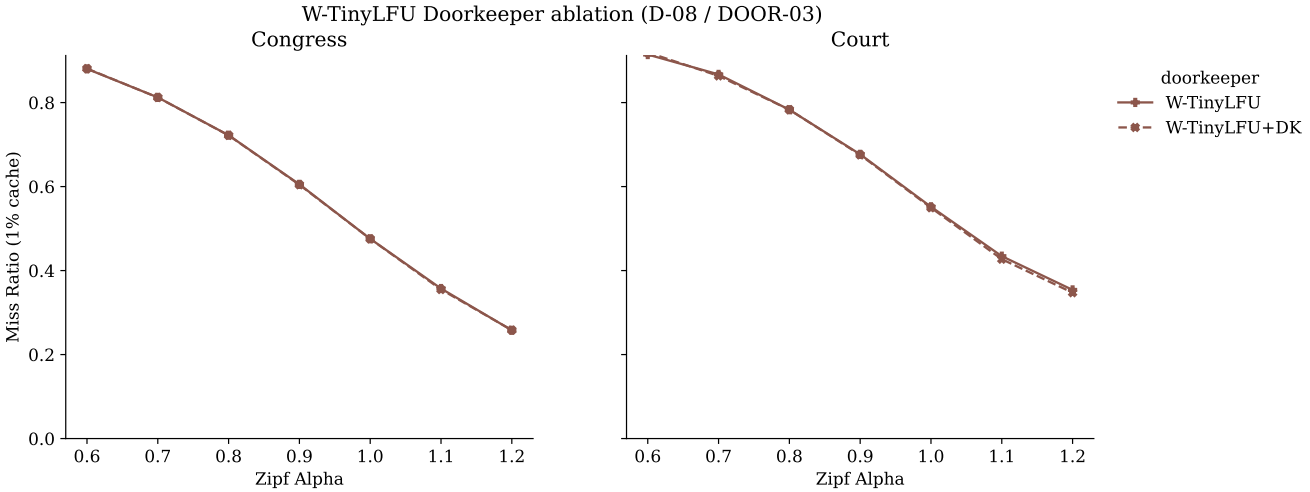


Figure 7: W-TinyLFU ± Doorkeeper ablation across α at cache_frac = 0.01. Shared brown color; solid = baseline W-TinyLFU, dashed = W-TinyLFU+DK variant. Congress panel shows essentially overlapping curves; Court panel shows a small direction-swap near the α extremes.

Finding: context-dependent with a small effect at the α extremes. Congress stays within ± 0.22 pp across every α (functionally noise). Court shows a peak -0.72 pp win for the +DK variant at $\alpha = 1.1$, but a $+0.52$ pp *loss* at $\alpha = 0.6$. **Interpretation:** Doorkeeper is a marginal hedge against one-hit- wonder spikes on a heavy-tailed workload, not a default. The direction-swap at low α confirms the Caffeine production team's decision to omit it — a workload without a real frequency gradient has nothing for Doorkeeper to absorb, and the filter's own false-positives become a small penalty.

Ablation takeaway

Reading the three ablations together: **SIEVE's lazy promotion is load-bearing** (the largest ablation delta in our study, +15.4 pp peak); **S3-FIFO benefits from a smaller small-queue than the SOSP'23 default** (actionable advice for practitioners on heavy-tailed workloads); and **Doorkeeper is context-dependent** (workload-sensitive hedge rather than a default). Together these results justify the claim that our policy ordering in §4 is stable across reasonable parameter variations within each policy — the ordering is a property of the policy families, not of the specific parameter choices we happened to ship in each implementation.

8 Heavy-Tailed Workloads Require CI Bands on Byte-MRC

The 462,490-byte Court document flagged in Table 1 (§3, max size vs. 1,381-byte median — a 335× outlier) dominates byte-miss-ratio analysis in ways that require methodological care. Byte-MRC on heavy-tailed

workloads behaves *qualitatively differently* from object-MRC, and single-seed benchmarks silently report this artifact as a policy effect.

8.1 The data

Table 4 reports 5-seed mean $\pm\sigma$ at `cache_frac=0.01` on Court for the three policies that span the performance spectrum.

Policy	Object miss-ratio	Byte miss-ratio
W-TinyLFU	0.728 ± 0.045	$\mathbf{0.611 \pm 0.207}$
SIEVE	0.783 ± 0.045	0.656 ± 0.205
LRU	0.847 ± 0.026	0.755 ± 0.171

Table 4: Court at `cache_frac=0.01`, 5-seed mean $\pm\sigma$. Object-miss σ is in the normal 0.03–0.05 range. Byte-miss σ is 4–7× larger — every policy's byte-miss swings by 20+ percentage points across seeds, swamping the 5pp inter-policy gap.

Object-miss σ is in the normal 0.03–0.05 range. Byte-miss σ is 4–7× larger. The gap between W-TinyLFU and LRU on byte-miss (14.4pp) is smaller than the seed-to-seed σ of any single policy — a single-seed run could easily report W-TinyLFU losing to LRU on byte-miss purely from seed variance.

8.2 The mechanism

Court's max object size is 462,490 bytes — 335× the 1,381-byte median. That single outlier represents:

- 0.78% of the working set by bytes (462 KB / 59.6 MB)
- 0.007% of the objects (1 of 15,018)

At `cache_frac=0.01` on Court (working set ≈ 59.6 MB), the cache is ≈ 600 KB. **That single outlier is ≈ 77% of the cache**. Whether it lives in cache for most of the trace is a near-binary signal on byte-miss-ratio. The replay-Zipf seed controls which key gets the highest popularity rank: if the seed assigns the 462 KB outlier rank-1 (highest popularity), it stays cached for the entire trace and byte-miss is low. If the seed assigns it rank-10,000+, it never enters cache and byte-miss is high. No eviction policy can fix this — it is a popularity-assignment artifact of the Zipf-over-raw-trace replay model, not a signal about eviction quality.

8.3 The paper claim

> *On a workload with a catastrophic size outlier, byte-level eviction metrics are dominated by whether the outlier object's Zipf rank is high enough to keep it resident. We report both object miss-ratio (policy-sensitive, low variance) and byte miss-ratio (policy-insensitive at this scale, high variance), and flag the difference as a feature of heavy-tailed workloads.*

Practitioner takeaway. Byte-miss benchmarks on traces with long tails can be misleading unless CI bands from multi-seed replay are reported. The standard practice of quoting a single-seed byte-miss number and comparing across policies fails silently here: the $\sigma = 0.207$ around W-TinyLFU's 0.611 mean is large enough that a single seed could report any value in $[0.40, 0.82]$ and still be within 1σ of the truth. Multi-seed CI bands (we used 5 seeds with Welch's t-test significance) are essential for heavy-tailed real-world workloads, not merely a matter of statistical rigor — they are the only way to distinguish a policy effect from a popularity-assignment artifact. Size-aware policies (LHD, AdaptSize, or W-TinyLFU with size-explicit admission) may reduce this variance on workloads with catastrophic size outliers; that extension is listed as V2-02 in §9.

9 Conclusion

We compared six cache eviction policies (LRU, FIFO, CLOCK, S3-FIFO, SIEVE, W-TinyLFU) on two real public-records API workloads — Congress.gov and CourtListener — using a 5-seed replay-Zipf model with Welch's t-test significance testing. W-TinyLFU wins 11 of 12 miss-ratio cells overall, but the headline finding is more nuanced than that count suggests: on Congress's near-uniform raw trace, SIEVE's visited-bit is functionally equivalent to W-TinyLFU's Count-Min Sketch across the entire α range from 0.6 to 1.5 (§5). The admission-filter benefit is workload-gated, not policy-inherent.

9.1 Winner per regime

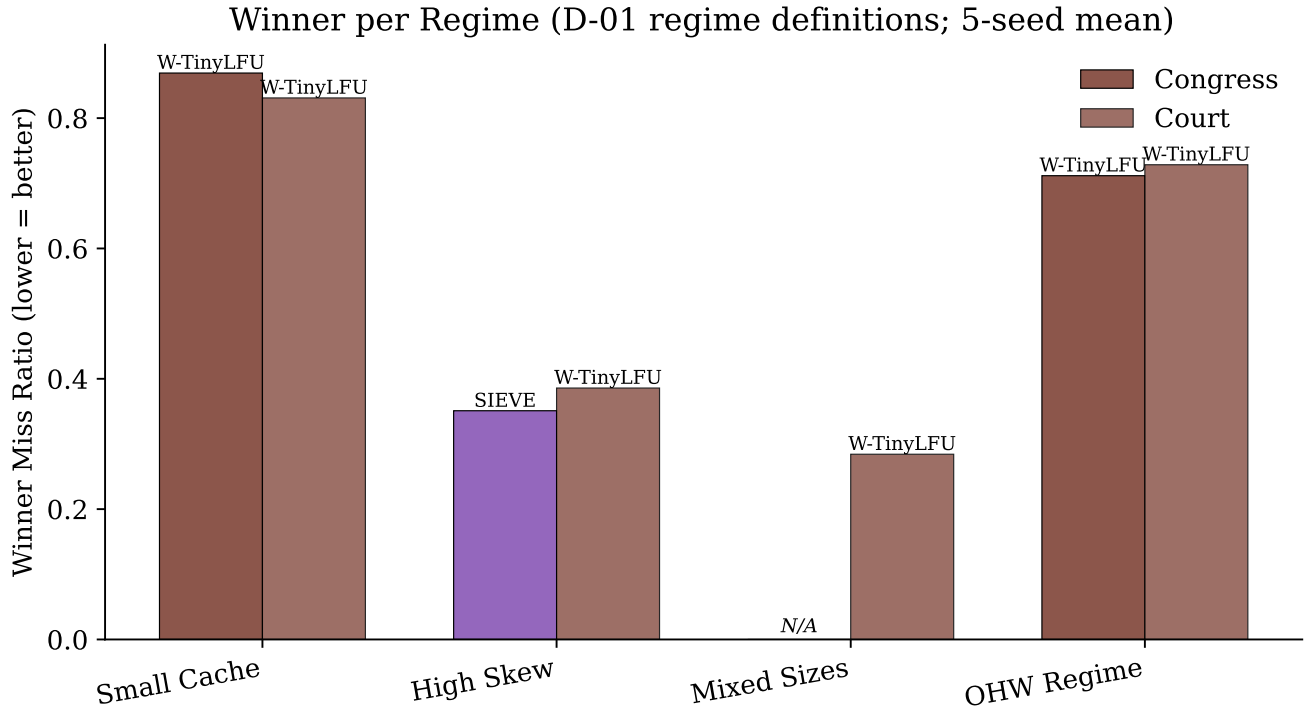


Figure 8: Winner-per-regime summary. Each bar shows the winning policy's miss-ratio (lower is better) in that regime on each workload. Base policies only; ablation variants excluded per §7.

Regime	Congress Winner	Cong Miss	Court Winner	Court Miss
Small Cache (`cache_frac=0.001`)	W-TinyLFU	0.869	W-TinyLFU	0.831
High Skew ($\alpha \in \{1.0, 1.1, 1.2\}$)	SIEVE	0.351	W-TinyLFU	0.386
Mixed Sizes (Court byte-MRC, single-seed)	N/A	—	W-TinyLFU	0.284
OHW Regime (`cache_frac=0.01`)	W-TinyLFU	0.712	W-TinyLFU	0.728

Table 5: Winner per regime across both workloads. Single-seed for the Mixed Sizes Court byte-miss-ratio row (multi-seed byte-MRC aggregation deferred to v2; see V2-05 below). Mixed Sizes on Congress is N/A because Congress has near-uniform object sizes (Table 1).

9.2 A practitioner decision tree

The most actionable takeaway from this study is a small flowchart for picking an eviction policy for a public-records API workload. Use this decision tree given a raw trace's characteristics:

- **If your raw trace has $\alpha_{\mathbf{raw}} < 0.5$ and a near-uniform size distribution (Congress-like):** SIEVE or W-TinyLFU are statistically tied across $\alpha \in [0.6, 1.5]$ (§5, Table 2). Prefer SIEVE for its simplicity — it has no admission filter, no frequency sketch, no Count-Min Sketch, just a visited-bit and a hand pointer. Implementation and operational-debugging cost is lower for identical miss-ratio performance.
- **If your raw trace has $\alpha_{\mathbf{raw}} > 0.8$ and a long-tail size distribution (Court-like): W-TinyLFU by $\sim$ 5pp at every α.** The Count-Min Sketch's frequency gradient extracts information SIEVE's binary visited-bit cannot. The implementation cost is real but modest (a CMS, a Doorkeeper Bloom filter, an admission-filter comparison); the performance gap is stable enough to justify it.
- **If you have a catastrophic size outlier (Court-like):** W-TinyLFU's admission filter matters most at small cache sizes, where a single one-hit-wonder admission can evict 5–10% of capacity. Report *both* object and byte miss-ratios with 5-seed CI bands; a single-seed byte-miss number is dominated by which Zipf rank the outlier lands at (§8).
- **Small caches amplify everything:** if `cache_frac` < 0.01, admission quality dominates (W-TinyLFU beats LRU by 9–13pp on both workloads in the Small Cache regime). If `cache_frac` > 0.1, all reasonable policies converge; eviction algorithm choice stops mattering.

9.3 Limitations and future work

Six items are acknowledged but deferred to a future revision. None of them changes any finding reported here; together they extend the rigor envelope.

- **Caffeine trace cross-validation (V2-01):** running our W-TinyLFU implementation on a Caffeine-published benchmark trace would provide a $\pm 2\%$ agreement check against the reference Java implementation [5]; deferred due to benchmark-trace access friction.
- **LHD or AdaptSize as a 7th policy (V2-02):** size-aware eviction would likely behave quite differently on Court's long-tail size distribution and would provide a direct counterfactual for the §8 outlier-variance story.
- **Third trace domain (V2-03):** a SEC EDGAR collector would complete the public-records trilogy and test whether the Congress-like / Court-like dichotomy generalizes or fragments across more diverse workload traits.
- **Multi-seed $\alpha \in \{1.3, 1.4, 1.5\}$ on Congress (V2-04):** the single-seed crossover data in §5 is suggestive but not CI-backed. Cheap to add (~ 30 s wall-clock per seed $\times$ 3 new α values $\times$ 5 seeds).
- **Multi-seed byte-MRC aggregation (V2-05):** the per-seed CSVs at `results/compare/multiseed/court/` already record `byte_miss_ratio`; the Phase 5 aggregator only consumes `miss_ratio`. A one-line change to `compare_workloads.py::aggregate` would retire the "single-seed" hedge on the Mixed Sizes row of Table 5.
- **Code review WR-01 / WR-02 (V2-06):** two latent-coupling warnings in `plot_results.py` (BASE_POLICIES filter missing on winner-bar; regime dispatch missing else clause). Neither manifests in the current pipeline; both would bite future work that adds ablation variants to the multi-seed sweep.

These items do not change any finding reported here; they extend the rigor envelope for future work.

A Supplementary Figures

This appendix provides the full figure set for reproducibility-minded readers. Main-body figures (Figures 1, 2, 3, 4, and the three §7 figures) carry the paper's argument; the figures below expand the same data with

per-workload detail, duplicate views used for different purposes in the main text, and the per-policy delta breakdowns that did not fit in the main body. All ten figures regenerate from a clean checkout via `make run-sweep` followed by `make plots WORKLOAD=congress` / `WORKLOAD=court` / `WORKLOAD=shards_large` and `python3 scripts/compare_workloads.py`.

A.1 Per-workload MRC, Congress

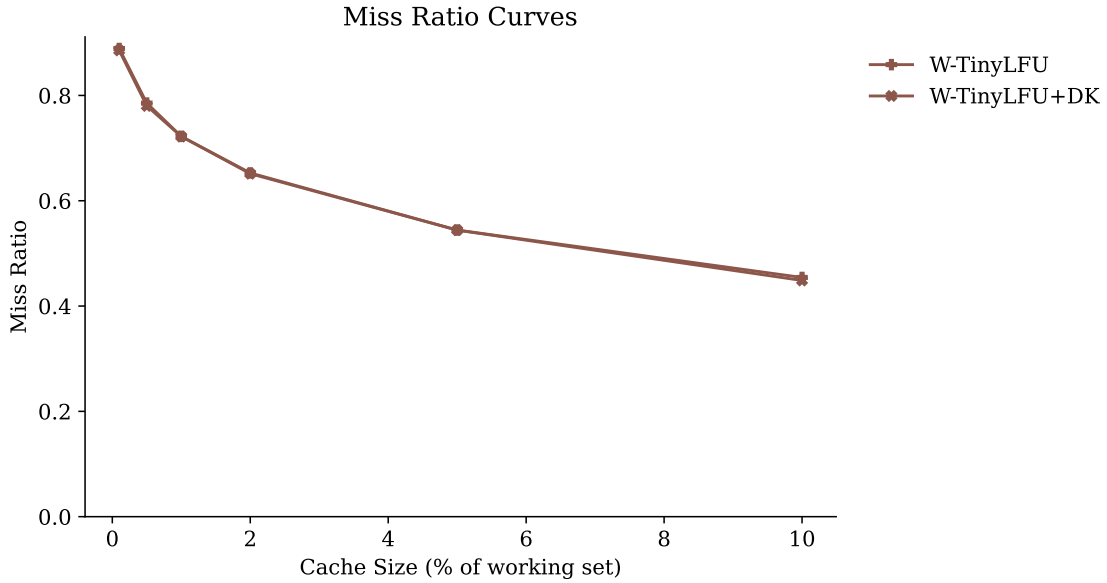


Figure 9: Congress miss-ratio curves across all six policies and the full cache-fraction sweep $\{0.001, 0.005, 0.01, 0.02, 0.05, 0.1\}$. Companion to the 2-panel main-body figure (Fig. 1) with the Congress panel expanded to show every sampled cache fraction.

A.2 Per-workload MRC, Court

A.3 Per-workload byte-MRC, Congress

A.4 Per-workload byte-MRC, Court

A.5 One-hit-wonder (OHW) breakdown, Congress

A.6 One-hit-wonder (OHW) breakdown, Court

A.7 SHARDS error profile (full)

A.8 SHARDS convergence (duplicated for reproducibility)

A.9 Per-policy Congress-Court delta

A.10 Cross-workload MRC overlay

References

[1] Y. Zhang et al., SIEVE is Simpler than LRU, *NSDI* '24.

[2] J. Yang et al., FIFO Queues are All You Need, *SOSP* '23.

[3] C. Waldspurger et al., SHARDS, *FAST* '15.

[4] G. Einziger et al., TinyLFU, *ACM ToS* '17.

[5] B. Manes, Caffeine v3.1.8 reference implementation.

[6] A. Clauset et al., Power-Law Distributions in Empirical Data, *SIAM Rev* '09.

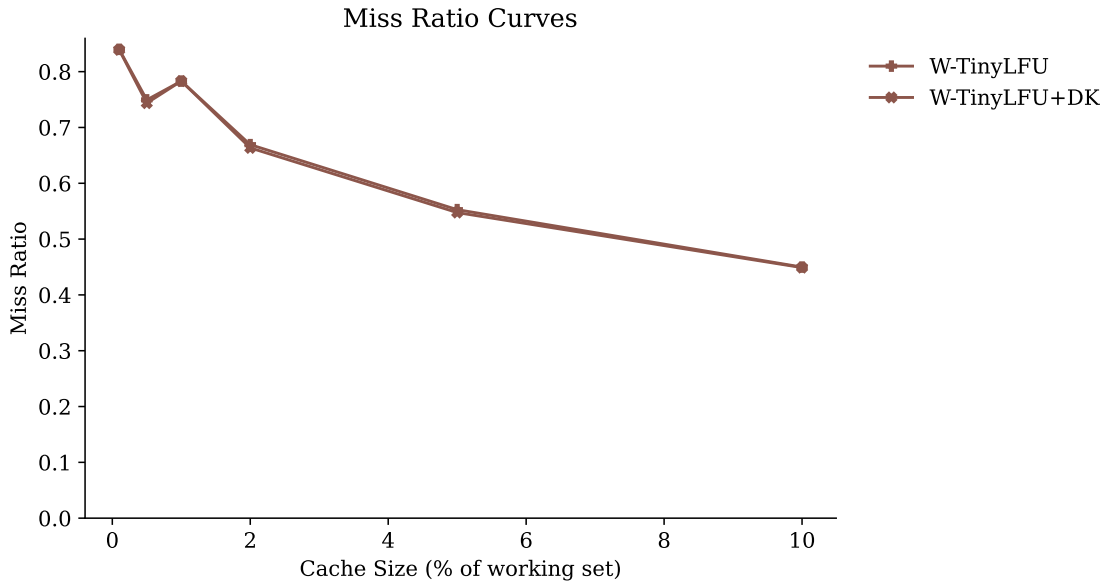


Figure 10: Court miss-ratio curves; same axes and policy color scheme as Fig. 9. Note the larger inter-policy spread at small cache fractions, reflecting Court's richer raw-trace locality structure (§3).

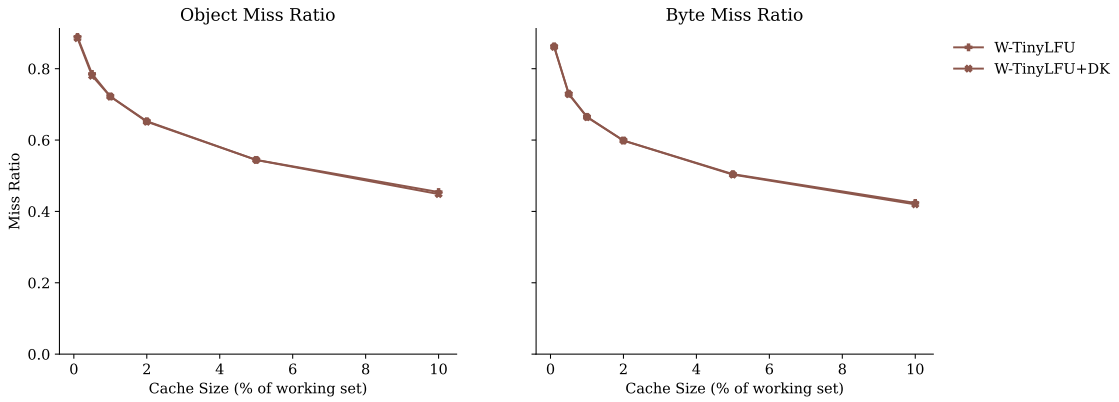


Figure 11: Congress byte-miss-ratio. Congress's near-uniform object sizes make the byte-MRC track the object-MRC closely — contrast with Court (Fig. 12) where the 462 KB outlier inflates seed-to-seed variance to $\sigma = 0.21$ (see §8).

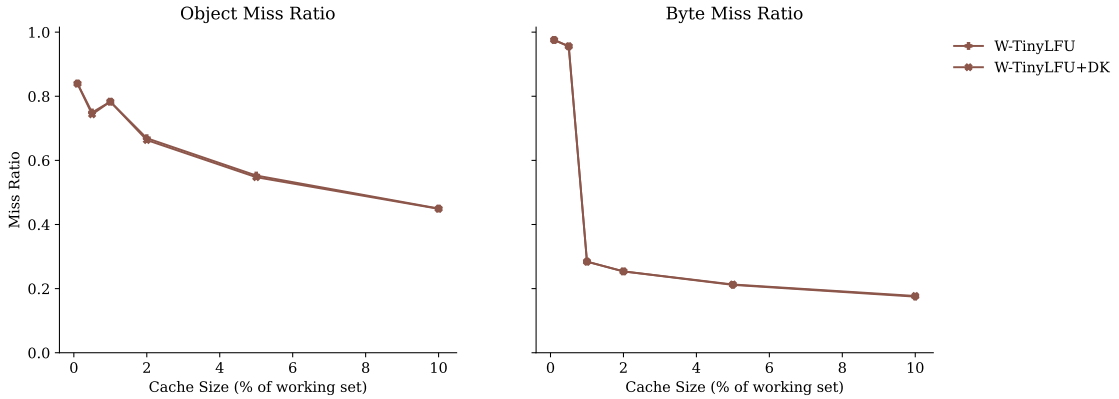


Figure 12: Court byte-miss-ratio. Note the wide seed-to-seed spread at small cache fractions — the single 462 KB outlier document dominates byte-miss analysis in ways detailed in §8.

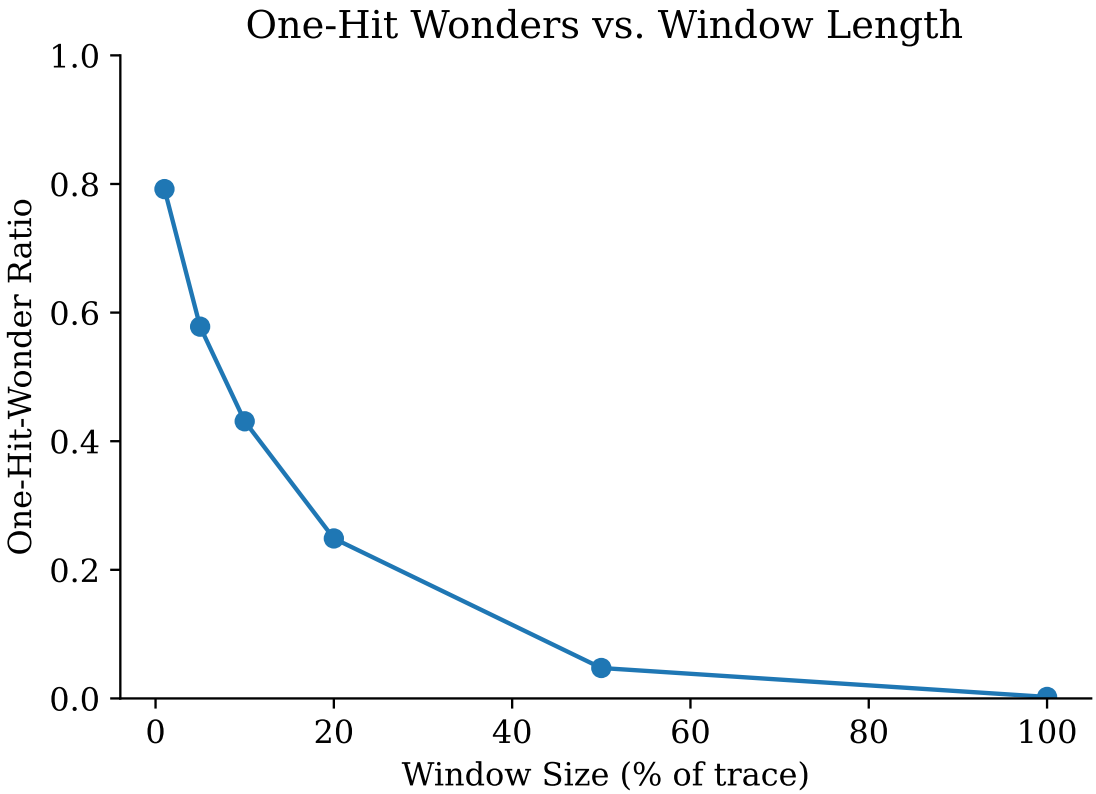


Figure 13: Congress one-hit-wonder ratio reaches 0.989 at `cache_frac=0.1` — the raw trace is dominated by keys accessed exactly once. This is the empirical motivation for the replay-Zipf overlay described in §3: without it, the raw Congress trace would be nearly unanalyzable for policy comparison.

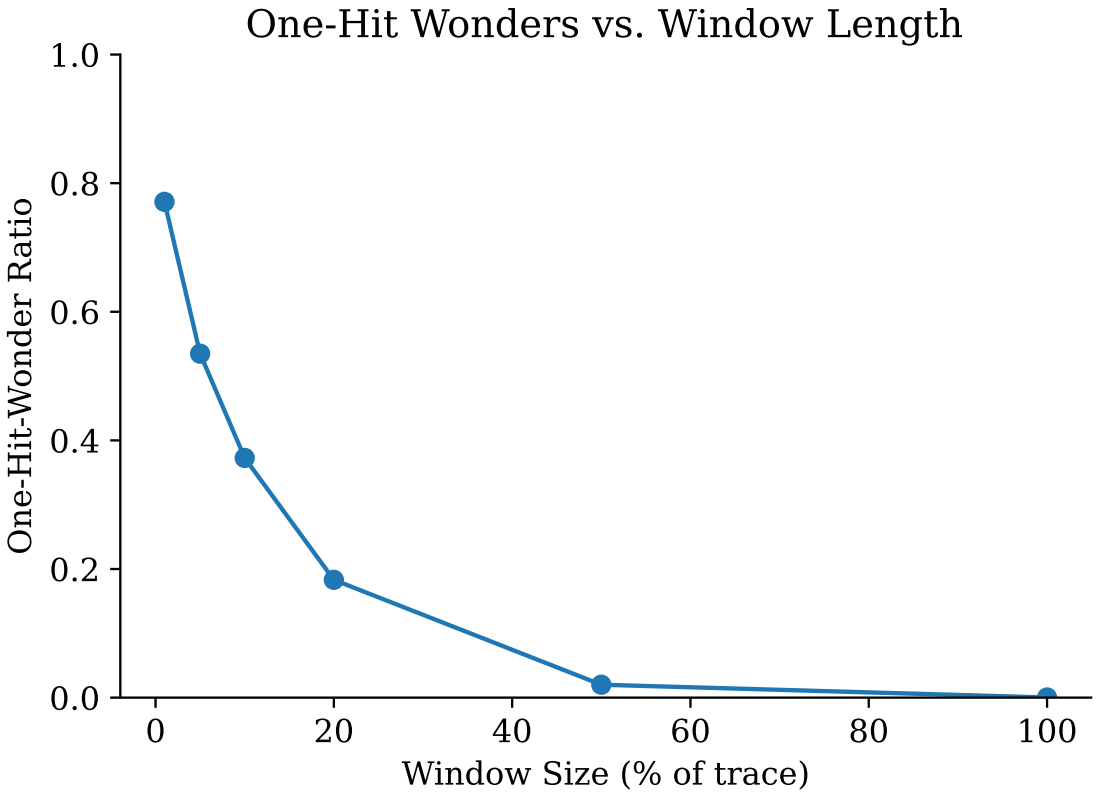


Figure 14: Court one-hit-wonder ratio reaches 1.000 at `cache_frac=0.1` — same replay-Zipf motivation as Congress, but Court's raw $\alpha_{\text{raw}} = 1.028$ (vs. Congress's 0.231) means the raw trace itself carries exploitable locality before the Zipf overlay is applied (§5).

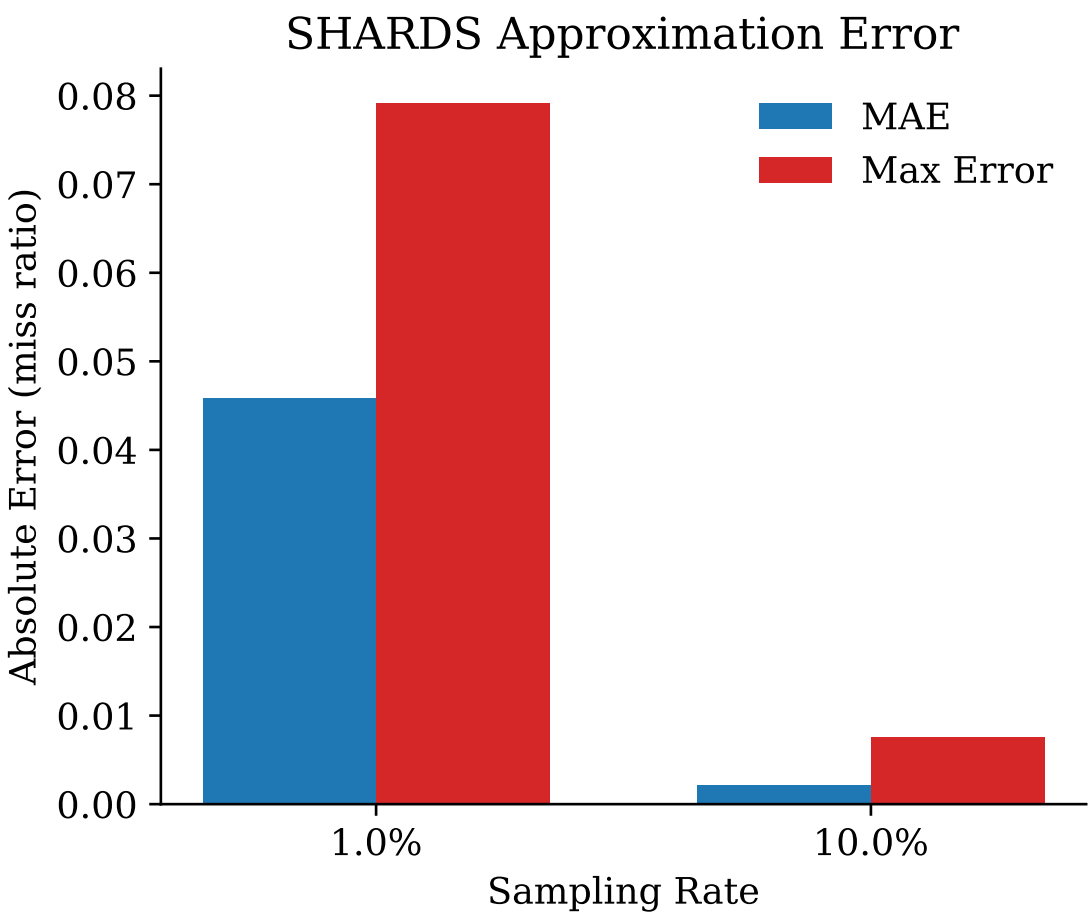


Figure 15: Full SHARDS error profile on the 1M-access trace across all sampling rates. Companion to the main-body convergence figure (Fig. 4); shows the full residual distribution rather than just the summary MAE / max-abs statistics.

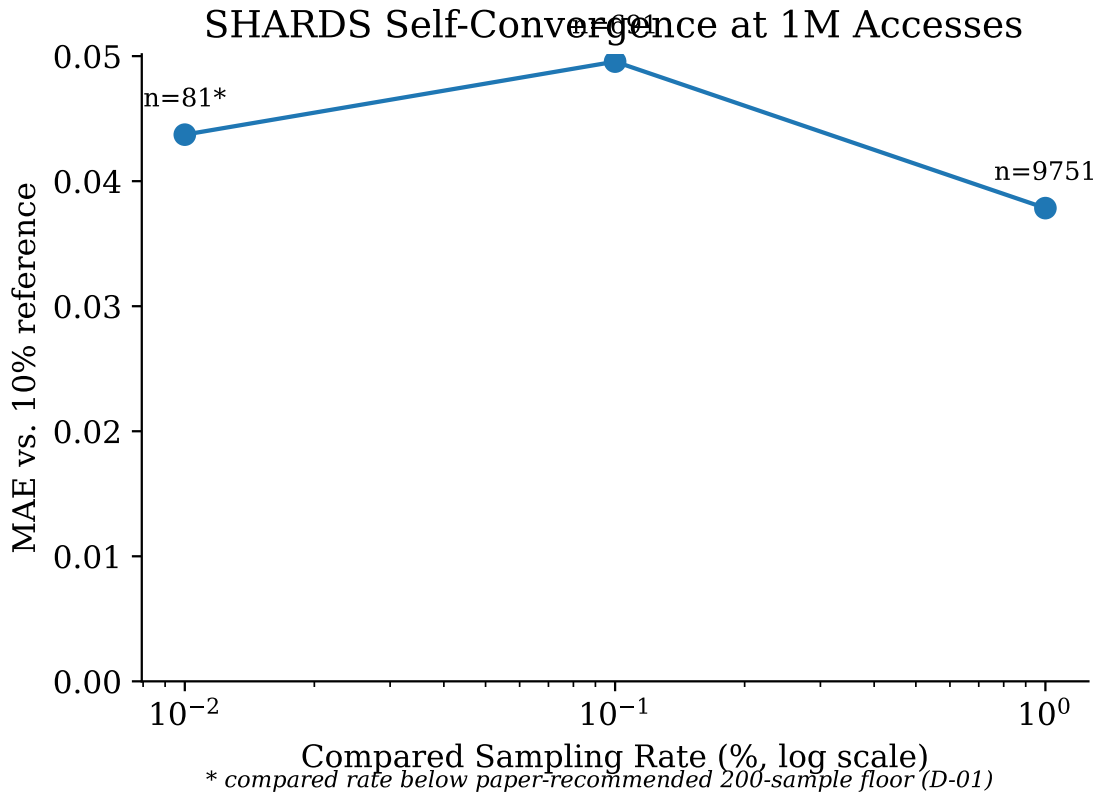


Figure 16: SHARDS mean-absolute-error and max-absolute error vs. sampling rate. Also shown in the main body (§6); reproduced here for self-contained appendix use so the reproducibility check in Fig. 15 has its reference baseline adjacent.

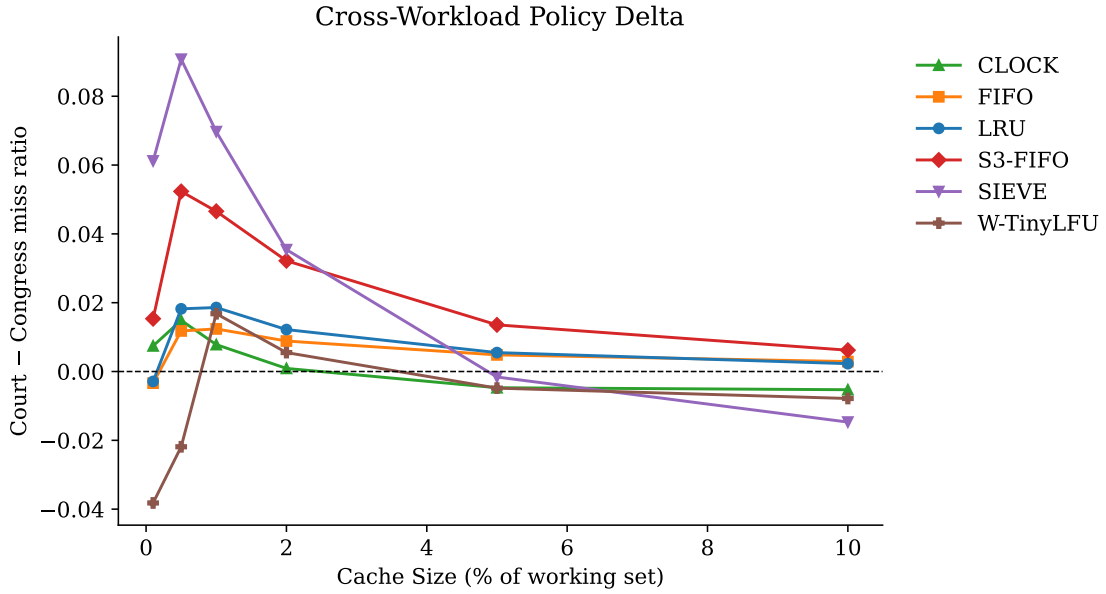


Figure 17: Per-policy miss-ratio delta (Court minus Congress) across cache fractions. Positive = policy does worse on Court. W-TinyLFU and LRU diverge most at small cache fractions — the admission filter earns its keep in the small-cache regime (§5, decision-tree rule 4 in §9).

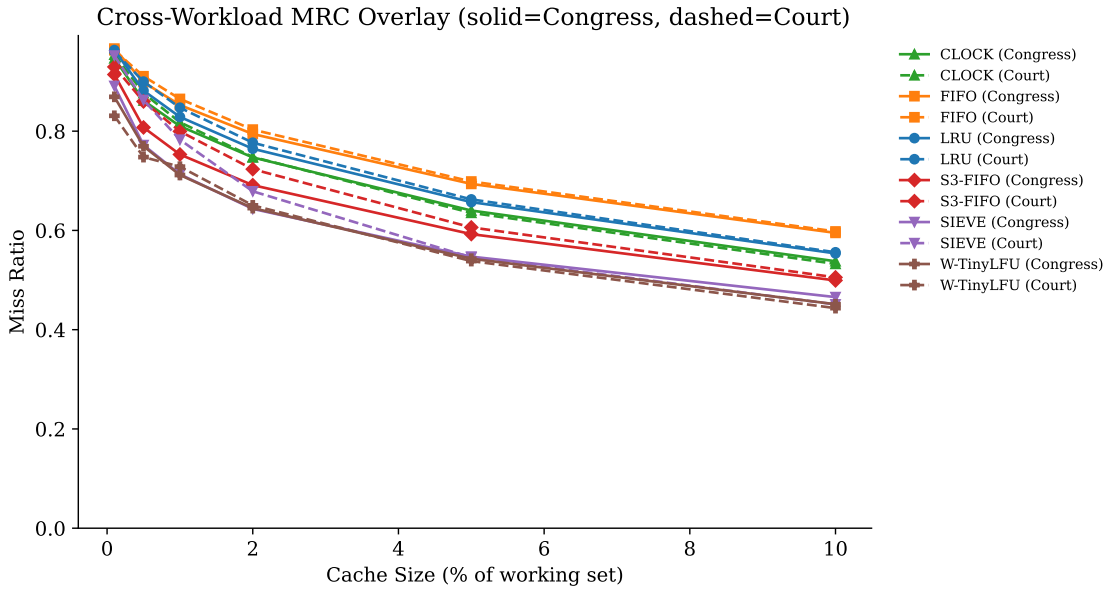


Figure 18: All six policies on both workloads in a single overlay — solid lines are Congress, dashed are Court. Collapses the 2-panel main-body figure (Fig. 1) into one panel for gestalt comparison; the W-TinyLFU≈SIEVE collapse on Congress and the 5pp W-TinyLFU lead on Court are both visible in a single glance.